NATIONAL UNIVERSITY OF MODERN LANGUAGES

SOFTWARE QUALITY ENGINEERING PROJECT

NAME: **M Abdul Rafay (FL23791)**

Nazish Khan (FL23843)

M Ali Atif (FL23824)

PROGRAM: BSSE (6th Semester)

COURSE: SOFTWARE QUALITY ENGINEERING

SUBMITTED TO: Mam Rubab

DATE: 27 May 2026 **DAY: Sunday**

Contents

List of Figures

1. Project Introduction

The Online Food Ordering System is a web-based application that allows users to order food from different restaurants. Customers can register, log in, browse restaurants, view menus, add items to the cart, place orders, make payments, and track order status.

The main purpose of this system is to provide an easy and efficient way for users to order food online. Restaurant admins can manage menus and orders, while system admins can control overall system operations.

This project focuses on applying Software Quality Engineering techniques to analyze, test, and improve the quality of the system.

2. System Scope and Features

The system includes all major functions from user registration to order delivery tracking.

Main Features

1. User registration and login
2. Restaurant listing
3. Menu browsing
4. Cart management
5. Order placement
6. Payment method selection
7. Order tracking
8. Admin management for restaurants, menus, and orders

Main Users

1. Customer
2. Restaurant Admin
3. System Admin

3. Quality Attributes Report

The following quality attributes are required for this system:

Quality Attribute	Description
Correctness	System should give correct results for all operations
Reliability	System should run without failure
Usability	Easy to use for all types of users
Performance	Fast response during all operations
Security	User data and payment information must be protected
Maintainability	Easy to update and fix errors in the code
Scalability	System should handle increasing number of users

Availability	System should always be accessible to users

4. Software Quality Plan

Objective: To ensure the system meets all requirements with good quality and minimum errors.

Quality Standards

- Correct functionality
- Secure system operations
- Easy user interface
- Fast performance
- Reliable output

Quality Activities

- Requirement analysis and review
- System design review
- Code review
- Testing activities
- Documentation review

Testing Types

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

5. Software Quality Model Mapping

Using McCall's Quality Model:

Quality Factor	How It Applies to This System
Correctness	System calculates and processes orders correctly
Reliability	System performs without failure
Efficiency	Fast processing of all requests
Integrity	Data is protected from unauthorized access
Usability	Easy interface for all users
Maintainability	Code is easy to modify and update
Flexibility	New features can be added easily
Portability	System can run on different platforms and devices

6. Software Metrics Report

Product Metrics

Lines of Code (LOC): Represents the total size of the software.

Modules in System:

#	**Module Name**
1	User Module
2	Restaurant Module
3	Menu Module
4	Cart Module
5	Order Module
6	Admin Module

Total Modules: 6

Defect Density

Formula: Defect Density = Number of Defects / KLOC

Example: If 10 defects are found in 1000 lines of code, Defect Density = 10 defects per KLOC

Cyclomatic Complexity

Function	**Cyclomatic Complexity**	**Meaning**
calculateOrderTotal	4	4 independent paths exist
updateOrderStatus	5	5 independent paths exist

Process Metrics

- Number of test cases executed
- Number of test cases passed
- Number of test cases failed
- Number of defects found
- Number of defects fixed

7. Goal Question Metric (GQM) Model

Goal: Improve the quality and reliability of the Online Food Ordering System.

#	**Question**	**Metric**
Q1	How many defects are found in the system?	Total number of defects found during testing

Q2	How many test cases are successful?	Pass Rate = (Passed Test Cases / Total Test Cases) x 100
Q3	How much time is required to fix defects?	Average defect fixing time
Q4	Is the order calculation function working correctly?	Number of correct outputs compared to incorrect outputs

8. Cost of Quality Analysis

Cost of Quality includes all costs related to preventing, detecting, and fixing defects.

Category	Description	Examples
Prevention Cost	Cost to prevent defects before they occur	Training, planning, requirement analysis
Appraisal Cost	Cost of checking and testing the system	Testing, inspections, reviews
Internal Failure Cost	Cost of fixing defects before release	Debugging, rework
External Failure Cost	Cost of defects found after release	Customer complaints, system fixes

Conclusion: Investing in prevention and appraisal costs reduces failures. It is always cheaper to find and fix defects early.

9. Review and Inspection Checklist

Requirement Review

#	Check Item	Status
1	Requirements are complete	Pass/Fail
2	Requirements are clear	Pass/Fail
3	Requirements are understandable	Pass/Fail
4	Requirements are testable	Pass/Fail

Design Review

#	Check Item	Status
1	System design is correct	Pass/Fail
2	Database design is proper	Pass/Fail
3	Security is considered	Pass/Fail

4	Error handling is included	Pass/Fail

Code Inspection

#	**Check Item**	**Status**
1	Code follows standards	Pass/Fail
2	Code is readable	Pass/Fail
3	Proper naming is used	Pass/Fail
4	No major errors exist	Pass/Fail
5	Comments are added in code	Pass/Fail

10. Software Testing Plan

Test Plan ID: TP-001

Project Name: Online Food Ordering System

Objective: To verify that all functions of the Online Food Ordering System work correctly according to the requirements and provide correct results to users.

Scope of Testing

- User registration and login
- Menu browsing and item selection
- Cart management
- Order placement and confirmation
- Payment method selection
- Order status tracking
- Order total calculation
- Order status update validation

Features Not to be Tested

- Database performance testing
- Network performance testing
- Third-party payment gateway internal operations
- Security penetration testing

Test Strategy

The following testing techniques will be used:

- Functional Testing
- Black-Box Testing (Equivalence Class Partitioning, Boundary Value Analysis)
- White-Box Testing (Path Coverage, Cyclomatic Complexity)

- Manual Testing

Entry and Exit Criteria

Criteria	Description
Entry	System requirements are approved, test environment is ready, code functions are available, test data is prepared
Exit	All planned test cases executed, critical defects fixed, test results documented, test summary completed

Risks and Mitigation

Risk	Mitigation
Requirement changes during testing	Review requirements regularly
Test environment failure	Maintain backup environment
Lack of test data	Prepare test data before testing
Limited testing time	Prioritize critical functionalities

Requirement Traceability Matrix (RTM)

Requirement ID	Description	Test Case ID
FR-01	User Registration	TC-01
FR-02	User Login	TC-02
FR-03	Cart Management	TC-03
FR-04	Order Placement	TC-04
FR-05	Payment Selection	TC-05
FR-06	Order Tracking	TC-06
FR-07	Order Total Calculation	TC-07, TC-08, TC-09
FR-08	Order Status Update	TC-10, TC-11, TC-12

11. Manual Test Cases

TC-01

Requirement ID	FR-01
Test Scenario	Register with valid user details

Test Steps	Enter name, email, and password and click Register
Test Data	Name: Rafay, Email: Rafay@gmail.com, Password: Rafay@123
Expected Result	Account should be created successfully
Actual Result	Account created successfully
Status	Pass

TC-02

Requirement ID	FR-02
Test Scenario	Login with correct credentials
Test Steps	Enter valid email and password and click Login
Test Data	Email: ali@gmail.com, Password: Ali@123
Expected Result	User should be logged in successfully
Actual Result	User logged in successfully
Status	Pass

TC-03

Requirement ID	FR-03
Test Scenario	Add item to cart
Test Steps	Select a food item and click Add to Cart
Test Data	Item: Burger, Price: Rs.500
Expected Result	Item should appear in the cart with correct price
Actual Result	Item added to cart correctly
Status	Pass

TC-04

Requirement ID	FR-04
Test Scenario	Place order with valid cart items
Test Steps	Add items to cart and click Place Order
Test Data	Cart with 2 items, total Rs.1000
Expected Result	Order should be placed and order ID generated
Actual Result	Order placed successfully
Status	Pass

TC-05

Requirement ID	FR-05
Test Scenario	Select payment method
Test Steps	Choose Cash on Delivery as payment method
Test Data	Payment Method: Cash on Delivery
Expected Result	Payment method should be saved successfully
Actual Result	Payment method saved
Status	Pass

TC-06

Requirement ID	FR-06
Test Scenario	Track order using order ID
Test Steps	Enter valid order ID in tracking section
Test Data	Order ID: ORD-1023
Expected Result	Current order status should be displayed
Actual Result	Order status displayed correctly
Status	Pass

TC-07

Requirement ID	FR-07
Test Scenario	Calculate order total with valid inputs, total above minimum
Test Steps	Call calculateOrderTotal with valid price, quantity, delivery and minimum order
Test Data	price=500, qty=2, delivery=50, minOrder=300
Expected Result	success: true, total=1000
Actual Result	success: true, total=1000
Status	Pass

TC-08

Requirement ID	FR-07
Test Scenario	Calculate order total when total is below minimum order
Test Steps	Call calculateOrderTotal with total below minimum order value
Test Data	price=100, qty=2, delivery=50, minOrder=300
Expected Result	success: true, total=250 (delivery charges added)
Actual Result	success: true, total=250
Status	Pass

TC-09

Requirement ID	FR-07
Test Scenario	Calculate order total with zero quantity
Test Steps	Call calculateOrderTotal with quantity set to zero
Test Data	price=200, qty=0, delivery=50, minOrder=300
Expected Result	success: false, message: Invalid quantity
Actual Result	success: false
Status	Pass

TC-10

Requirement ID	FR-08
Test Scenario	Update order status from pending to confirmed
Test Steps	Call updateOrderStatus with currentStatus=pending, newStatus=confirmed
Test Data	currentStatus: pending, newStatus: confirmed
Expected Result	Order confirmed successfully
Actual Result	Order confirmed successfully
Status	Pass

TC-11

Requirement ID	FR-08
Test Scenario	Update a cancelled order
Test Steps	Call updateOrderStatus with currentStatus set to cancelled
Test Data	currentStatus: cancelled, newStatus: confirmed
Expected Result	Cancelled order cannot be updated
Actual Result	Cancelled order cannot be updated
Status	Pass

TC-12

Requirement ID	FR-08
Test Scenario	Reverse transition from delivered back to pending
Test Steps	Call updateOrderStatus with currentStatus=delivered, newStatus=pending
Test Data	currentStatus: delivered, newStatus: pending
Expected Result	Transition should not be allowed

Actual Result	Order is pending (transition allowed - Bug)
Status	Fail

Test Summary

Category	Total Test Cases	Passed	Failed
System-Level (TC-01 to TC-06)	6	6	0
calculateOrderTotal (TC-07 to TC-09)	3	3	0
updateOrderStatus (TC-10 to TC-12)	3	2	1
Total	**12**	**11**	**1**

12. Black-Box Testing Report

Black-box testing tests the function based only on inputs and outputs without looking at internal code.
Techniques Used: Equivalence Class Partitioning and Boundary Value Analysis

Function: calculateOrderTotal

Equivalence Class Partitioning (ECP):

Equivalence Class	Condition	Type
EC1	quantity <= 0	Invalid
EC2	quantity > 0	Valid
EC3	itemPrice <= 0	Invalid
EC4	itemPrice > 0	Valid
EC5	total < minimumOrder	Valid (delivery added)
EC6	total >= minimumOrder	Valid (no delivery)

Boundary Value Analysis (BVA):

TC ID	Input Field	Test Value	Expected Result
TC-B1	quantity	0	Invalid quantity
TC-B2	quantity	1	Valid
TC-B3	itemPrice	0	Invalid item price
TC-B4	itemPrice	1	Valid
TC-B5	total vs minOrder	total=299, min=300	Delivery charges added
TC-B6	total vs minOrder	total=300, min=300	No delivery charges

TC-B7	total vs minOrder	total=301, min=300	No delivery charges

Function: updateOrderStatus

Decision Table:

currentStatus	newStatus	Expected Result
pending	confirmed	Order confirmed successfully
confirmed	delivered	Order delivered successfully
cancelled	anything	Cancelled order cannot be updated
delivered	pending	Should not be allowed (Bug found)
any	invalid value	Invalid order status

13. White-Box Testing Report

White-box testing looks inside the code and tests every path, branch, and condition.

Function: calculateOrderTotal

Control Flow:

START

-> Check quantity <= 0 -> YES -> return failure (Invalid quantity)

-> Check itemPrice <= 0 -> YES -> return failure (Invalid item price)

-> Calculate total = itemPrice x quantity

-> Check total < minimumOrder -> YES -> add delivery charges

-> return success with total

END

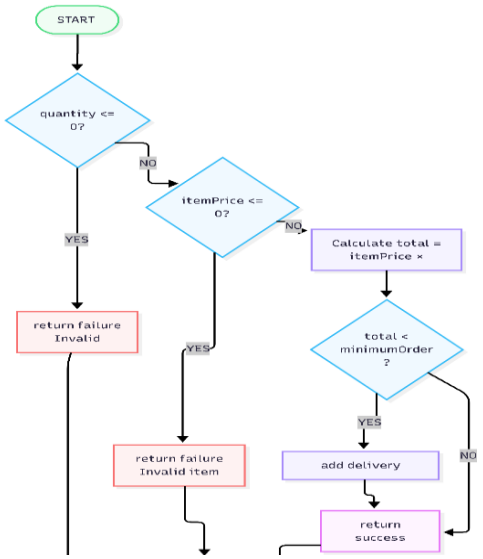

Figure 1: Control Flow Diagram of Calculate total order

Cyclomatic Complexity:
V(G) = Number of Decision Nodes + 1 = 3 + 1 = 4
Therefore, 4 independent paths are required for complete coverage.

Independent Paths:

Path	Description
Path 1	quantity <= 0 -> return failure (Invalid quantity)
Path 2	itemPrice <= 0 -> return failure (Invalid item price)
Path 3	Valid inputs, total < minimum -> delivery charges added
Path 4	Valid inputs, total >= minimum -> no delivery charges

Path Coverage Test Cases:

TC ID	qty	price	delivery	minOrder	Expected Result
PTC-01	2	500	50	300	success: true, total=1000 (Path 4)
PTC-02	2	100	50	300	success: true, total=250 (Path 3)
PTC-03	0	200	50	300	success: false, Invalid quantity (Path 1)
PTC-04	2	0	50	300	success: false, Invalid item price (Path 2)

Function: updateOrderStatus

Control Flow:
START
-> Check currentStatus === cancelled -> YES -> return message
-> Check newStatus === pending -> YES -> return message
-> Check newStatus === confirmed -> YES -> return message
-> Check newStatus === delivered -> YES -> return message
-> return Invalid order status
End

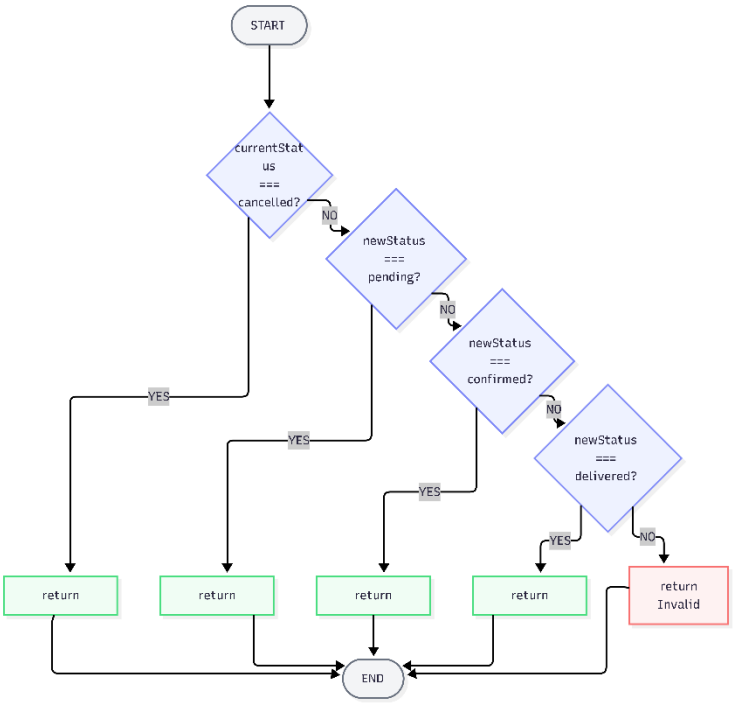

Figure 2: Control Flow Diagram of Update Order Status

Cyclomatic Complexity:
V(G) = Number of Decision Nodes + 1 = 4 + 1 = 5
Therefore, 5 independent paths are required for complete coverage.

Independent Paths:

Path	**Description**
Path 1	currentStatus = cancelled == cannot update
Path 2	newStatus = pending == order is pending
Path 3	newStatus = confirmed == order confirmed successfully
Path 4	newStatus = delivered == order delivered successfully
Path 5	None match == invalid order status

Path Coverage Test Cases:

TC ID	**currentStatus**	**newStatus**	**Expected Result**
PTC-01	pending	confirmed	Order confirmed successfully (Path 3)
PTC-02	confirmed	delivered	Order delivered successfully (Path 4)

PTC-03	cancelled	confirmed	Cancelled order cannot be updated (Path 1)
PTC-04	confirmed	pending	Order is pending (Path 2)
PTC-05	confirmed	cooking	Invalid order status (Path 5)

Branch Coverage Summary

Function	Total Branches	Branches Covered	Coverage %
calculateOrderTotal	4	4	100%
updateOrderStatus	5	5	100%

Statement Coverage: 100%. All statements in both functions are executed by the test cases.

Bugs Found Through White-Box Testing

#	Bug	Location	Severity
1	No null check for inputs	Both functions	High
2	Reverse transition not blocked (delivered to pending allowed)	updateOrderStatus	High
3	Negative delivery charges not rejected	calculateOrderTotal	Medium
4	No comments or documentation in code	Both functions	Low
5	Missing statuses: preparing, out for delivery	updateOrderStatus	Medium

Final Conclusion

This project applied Software Quality Engineering techniques to the Online Food Ordering System. The system was analyzed and tested using black-box testing, white-box testing, and manual testing methods.

A total of 12 manual test cases were executed. 11 passed and 1 failed. The failed test case revealed a real bug in the code where a delivered order can incorrectly go back to pending status.

Other bugs found include: negative delivery charges are not rejected, null inputs are not handled properly, and some order statuses like preparing are missing from the update function.

Software quality practices helped in identifying these errors early, which reduces the cost and effort of fixing them later. Overall, the system works correctly for most cases but requires improvements in input validation and status transition logic.